

Ch. 10

# 函式與方法

Horazon

C#程式設計

# 為什麼需要函式？

在撰寫程式時，我們經常會遇到**重複的邏輯**：

```
// 計算圓 A 面積
double areaA = 3.14 * 5 * 5;
Console.WriteLine("Area A: " + areaA);

// 計算圓 B 面積
double areaB = 3.14 * 10 * 10;
Console.WriteLine("Area B: " + areaB);
```

這樣的寫法有兩個缺點：

1. **重複程式碼**：難以維護，如果要改公式 (例如 3.14159)，要改很多地方。
2. **可讀性差**：無法一眼看出這段程式在做什麼 (雖然這個例子很簡單)。

# 函式 (Function) / 方法 (Method)

將一段**特定功能**的程式碼包裝起來，並給它一個**名字**。

- **重用性 (Reusability)**：寫一次，用很多次。
- **抽象化 (Abstraction)**：隱藏實作細節，只關心功能。
- **模組化 (Modularity)**：將大程式拆解成小零件。

在 C# (物件導向語言) 中，定義在類別內的函式通常稱為 **方法 (Method)**。

# 語法結構

```
回傳型別 方法名稱 (參數列)
{
    // 方法本體 (要做的事情)

    return 回傳值;
}
```

- **回傳型別 (Return Type)**：執行完後會給出什麼資料？(如 `int` , `string` , `bool` )。若不回傳任何東西，則用 `void` 。
- **方法名稱 (Method Name)**：通常使用動詞開頭，PascalCase (如 `CalculateArea` )。
- **參數列 (Parameters)**：輸入給方法的資料。

# 範例：定義一個加法方法

```
// 定義方法
int Add(int a, int b)
{
    int result = a + b;
    return result; // 回傳結果
}
```

```
// 呼叫方法 (Call)
int sum = Add(5, 3);
Console.WriteLine(sum); // 8
```

# 範例：沒有回傳值 (void)

如果方法只是要「做一件事」而不需要產出結果，型別使用 `void`。

```
void SayHello(string name)
{
    Console.WriteLine($"Hello, {name}!");
    // 不需要 return，除非想提早結束
}
```

```
SayHello("Horazon"); // 輸出: Hello, Horazon!
```

# 參數 (Parameters) 與 引數 (Arguments)

- 參數 (Parameters)：定義方法時的變數 (如 `int a`, `int b`)。
- 引數 (Arguments)：呼叫方法時實際傳入的值 (如 `5`, `3`)。

```
void PrintInfo(string name, int age)
{
    Console.WriteLine($"{name} is {age} years old.");
}

// 呼叫時順序要對應
PrintInfo("Alice", 20);
```

# 變數的作用域 (Scope)

在方法內宣告的變數，稱為**區域變數 (Local Variable)**。  
它們**只在該方法內有效**，方法結束後就會消失。

```
void Test()  
{  
    int x = 10; // x 只有在 Test 裡面活著  
}  
  
void Main()  
{  
    // Console.WriteLine(x); // 錯誤！這裡找不到 x  
}
```

不同的方法可以有同名的區域變數，它們互不影響。



# 實戰練習 1

請撰寫一個方法 `CalculateCircleArea`，輸入半徑，回傳面積。

```
double CalculateCircleArea(double radius)
{
    return 3.14159 * radius * radius;
}

// 主程式
double area = CalculateCircleArea(5.0);
Console.WriteLine($"圓面積: {area}");
```

這樣一來，計算公式就統一管理了！

# 實戰練習 2：BMI 計算機

試著寫一個計算 BMI 的方法：

BMI = 體重(kg) / 身高平方(m<sup>2</sup>)

```
double CalculateBMI(double weight, double heightCm)
{
    double heightM = heightCm / 100.0;
    return weight / (heightM * heightM);
}

double myBMI = CalculateBMI(70, 175);
Console.WriteLine($"BMI: {myBMI}");
```

# return 的特性 (1)：回傳數值

`return` 關鍵字最主要的作用是**回傳運算結果**給呼叫者。

一旦執行到 `return`，方法就會帶著結果**立即返回**，後面的程式碼不會被執行。

```
string CheckScore(int score)
{
    if (score >= 60)
        return "及格";
    else
        return "不及格";
}
```

## return 的特性 (2)：立即結束 (void)

在 `void` (不回傳值) 的方法中，我們可以使用 `return;` (不帶值) 來**強制結束**方法的執行。通常用於**排除異常狀況**，稱為 **Early Return (提早離開)**。

```
void Heal(int amount)
{
    // 1. 檢查無效狀況 (Guard Clause)
    if (amount <= 0)
    {
        Console.WriteLine("補血量無效!");
        return; // 遇到 return 直接結束，下面不會執行
    }

    // 2. 執行正常邏輯
    Console.WriteLine($"恢復了 {amount} 點生命值");
}
```

# 方法呼叫方法 (Method Calling Method)

方法不只能被 `Main` 呼叫，也可以**呼叫其他方法**。

透過層層呼叫，我們可以將複雜的任務拆解成簡單的小步驟。

```
void Cook()
{
    BoilWater();    // 呼叫煮水
    AddNoodles();   // 呼叫放麵
    Console.WriteLine("麵煮好了!");
}

void BoilWater()
{
    Console.WriteLine("水滾了...");
}

void AddNoodles()
{
    Console.WriteLine("放入麵條...");
}
```

# 進階概念：遞迴 (Recursion)

方法不只可以被別人呼叫，還可以**呼叫自己**！  
這稱為 **遞迴 (Recursion)**。

就像是俄羅斯娃娃，一層一層打開，直到最後一個 (終止條件)。

## 遞迴的兩大關鍵

1. **終止條件 (Base Case)**：什麼時候停下來？(沒有這個會變成無窮迴圈)
2. **遞迴步驟 (Recursive Step)**：呼叫自己，但問題規模變小。

# 遞迴範例：計算次方 (Power)

計算 2 的 3 次方： $2^3 = 2 * 2 * 2$

## 方法 1：使用遞迴

```
int Power(int baseNum, int exp)
{
    // 1. 終止條件：任何數的 0 次方都是 1
    if (exp == 0) return 1;

    // 2. 遞迴步驟： $2^3 = 2 * 2^2$ 
    return baseNum * Power(baseNum, exp - 1);
}
Console.WriteLine(Power(2, 3)); // 8
```

## 方法 2：使用內建函式庫 (更常用)

```
double result = Math.Pow(2, 3); // 需回傳 double
Console.WriteLine(result);      // 8
```

# 綜合練習 1：溫度轉換

請撰寫一個方法 `CtoF`，將攝氏溫度轉為華氏。

公式： $F = C * 1.8 + 32$

```
double CtoF(double c)
{
    return c * 1.8 + 32;
}

Console.WriteLine(CtoF(25)); // 77
```



# 綜合練習 2：比大小

請撰寫一個方法 `GetMax`，傳入兩個整數，回傳比較大的那個。

```
int GetMax(int a, int b)
{
    if (a > b) return a;
    else return b;
}

// 或是使用更簡潔的寫法
// int GetMax(int a, int b) => (a > b) ? a : b;

Console.WriteLine(GetMax(10, 20)); // 20
```

# 綜合練習 3：印出星星塔

請撰寫一個方法 `PrintStars`，輸入層數，印出對應的星星塔。

```
void PrintStars(int rows)
{
    for (int i = 1; i <= rows; i++)
    {
        for (int j = 0; j < i; j++)
        {
            Console.Write("*");
        }
        Console.WriteLine(); // 換行
    }
}

PrintStars(3);

//*
//**
//***
```

# 總結

- **方法 (Method)** 用來封裝重複的邏輯，提高程式可讀性與維護性。
- **回傳型別**決定方法產出的資料類型，`void` 代表不回傳。
- **參數**是方法的輸入，讓方法更具彈性。
- **return** 用來回傳值並結束方法。
- 變數有其**作用域**，方法內的變數外面看不到。
- **遞迴**是方法呼叫自己的技巧，需注意設定終止條件。